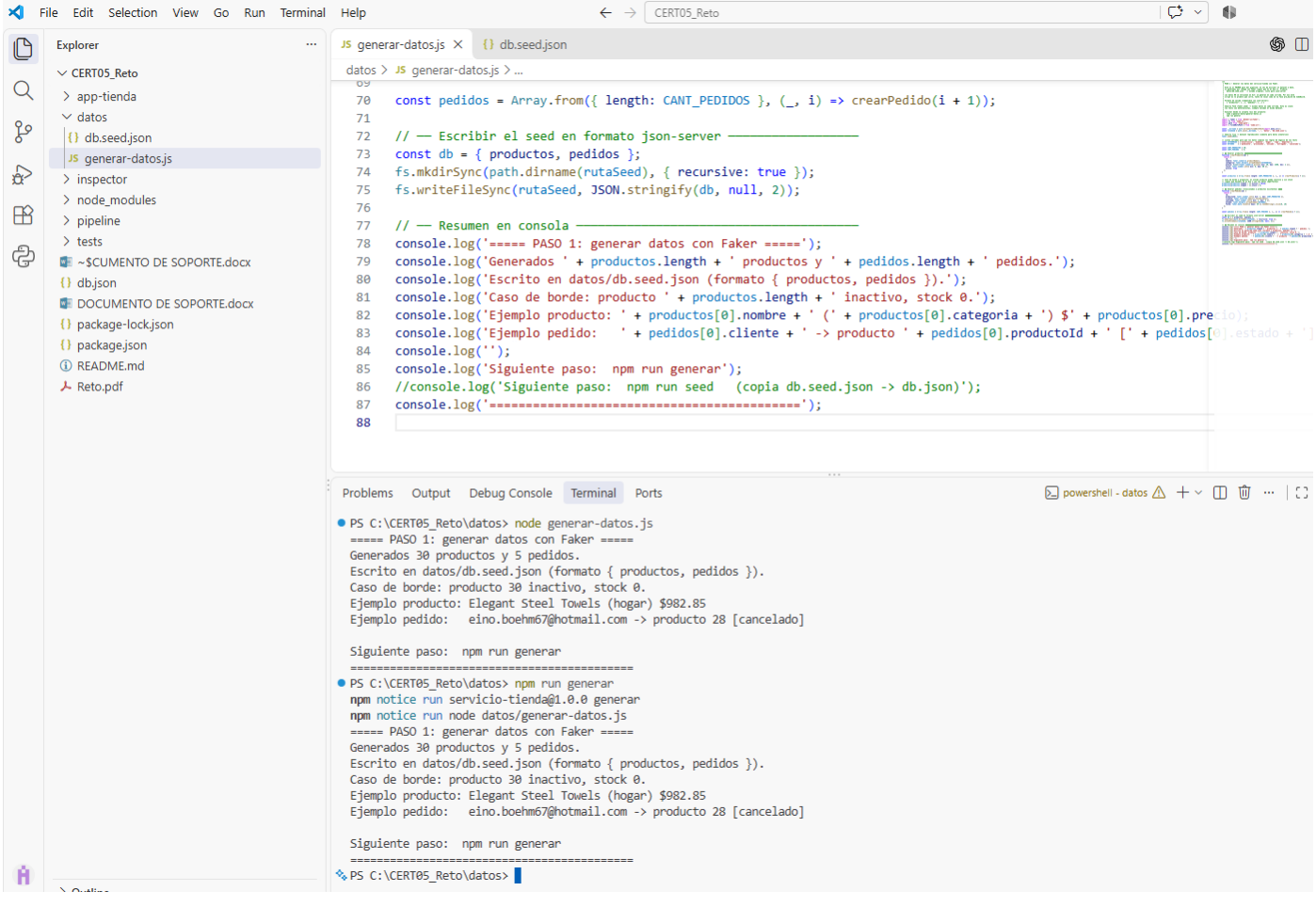
Módulo 5 — QA Cloud-Native: Automatización Inteligente con IA Generativa

Randolph Núñez Ramirez.

GitHub:

Repositorio de la solución.

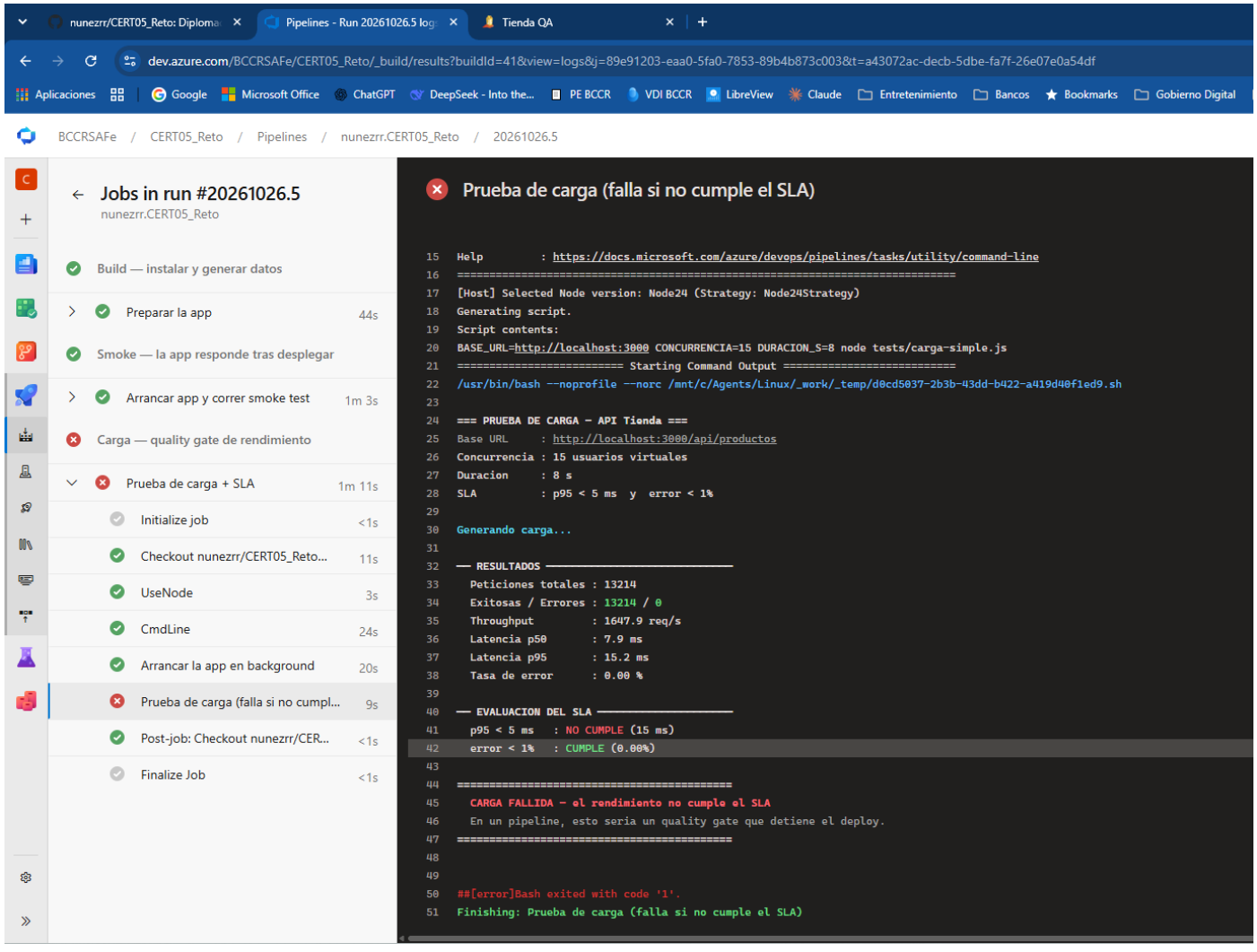
Fase 1: generar-datos.js (semilla fija) + db.json.

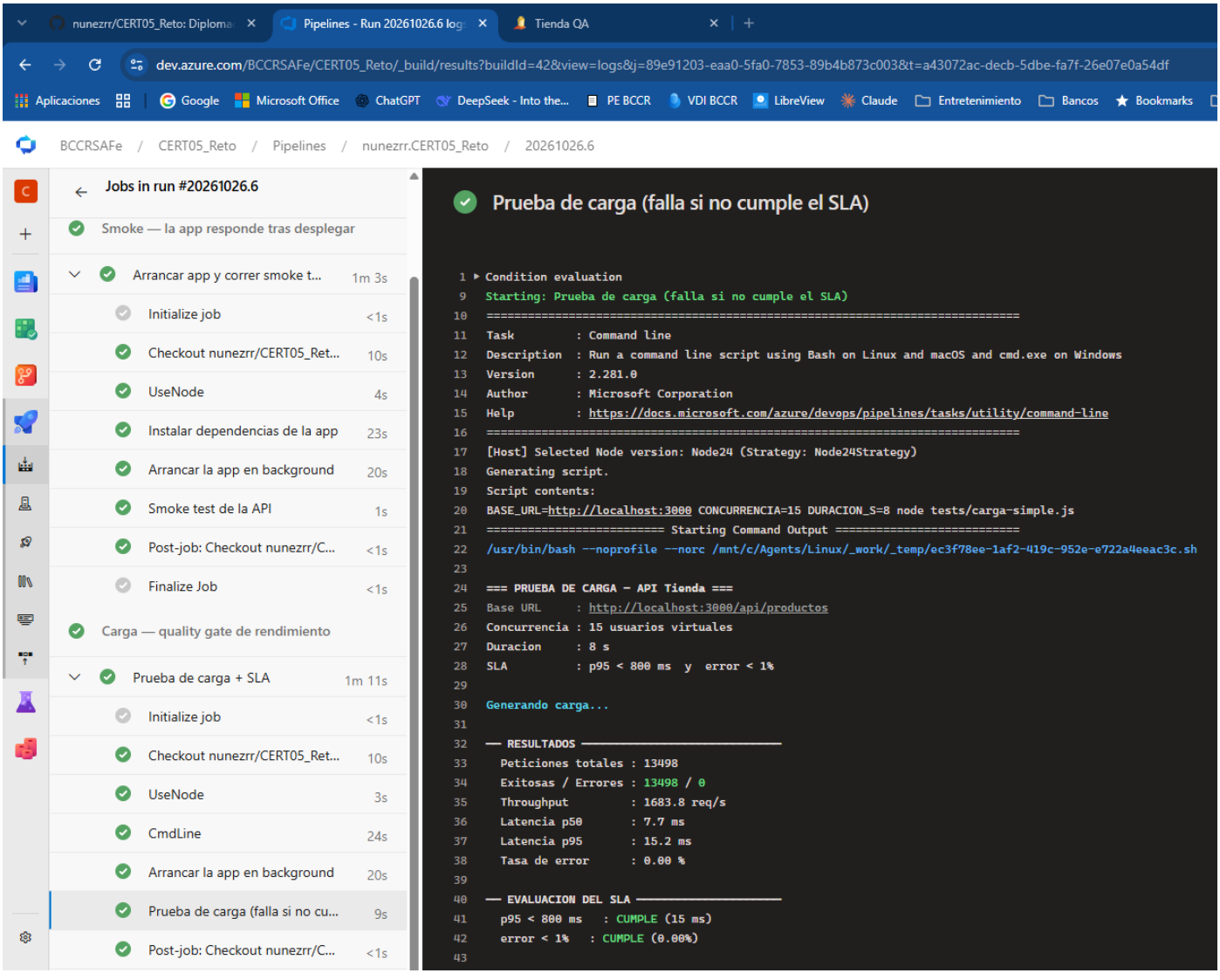


Explicar por qué NO versionas datos generados (los regenera el pipeline).

Cada vez que se corre el script se obtiene los mismos datos. Si una prueba falla puedes generar el mismo dataset y reproducir el fallo.

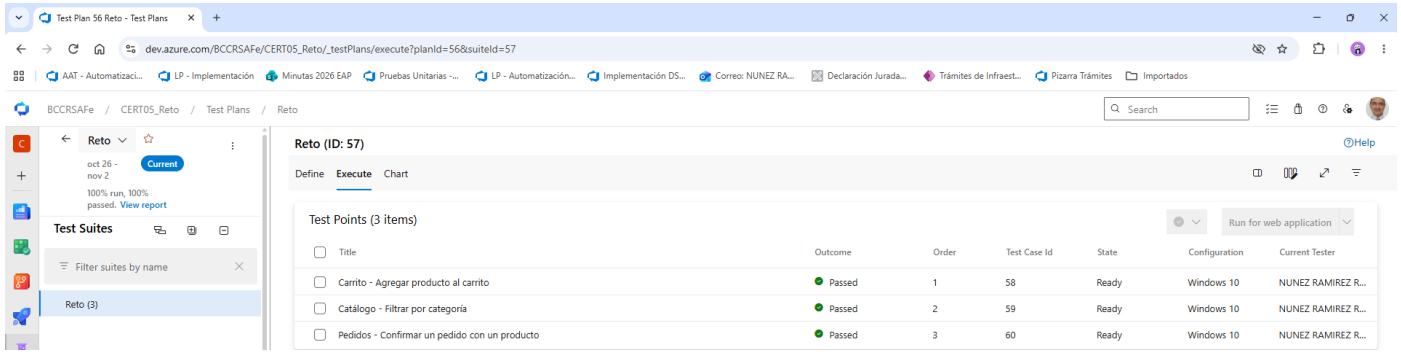
Fase 2: azure-pipelines.yml + capturas del gate de carga en rojo y en verde.

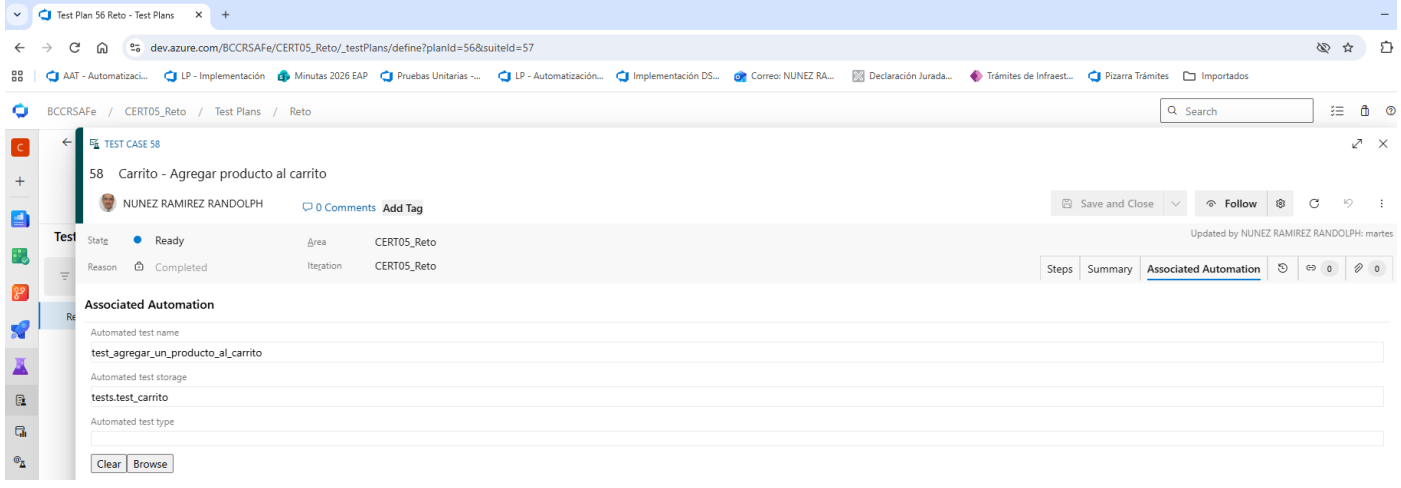


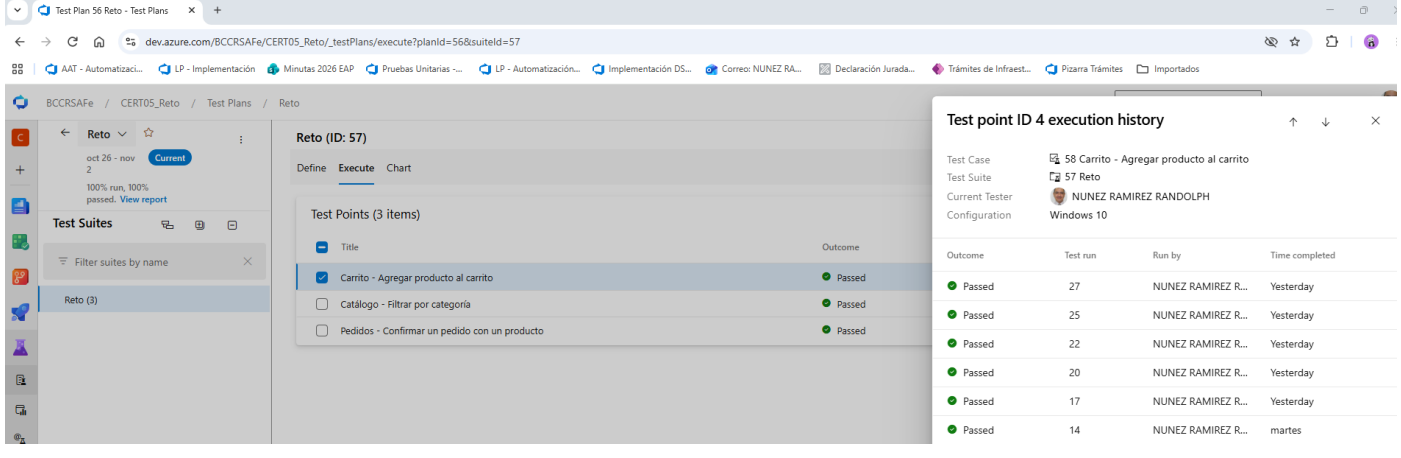


Uso Interno estricto

Fase 3: features/ y steps/ + salida de pytest en verde + escenario generado con IA (con el prompt).







1. Abre la URL http://localhost:3000/
2. Lee los siguientes escenarios que se encuentran en la ubicación: CERT05_Reto\e2e\features
3. Propón un escenario (un caso de borde) de prueba en Gherkin no contemplado en el paso anterior.
4. Genera un nuevo archivo en la ubicación del paso 2 (siguiendo el formato de los archivos existentes).
5. Revisa la carpeta: CERT05_Reto\e2e
6. Especifica paso a paso (archivos nuevos o modificados, código) que se deben aplicar para construir el nuevo escenario en la ruta anterior (tomando en cuenta lo ya creado).

Completed 22 steps in 1m 54s +59 -2

Revisé la tienda en `http://localhost:3000/` y los escenarios existentes de catálogo, carrito y pedidos. El caso de borde no cubierto es **buscar un texto que no coincide con ningún producto**. En la aplicación, ese caso debe dejar cero tarjetas y mostrar "No hay productos para ese filtro."; confirmé ambas cosas en el navegador.

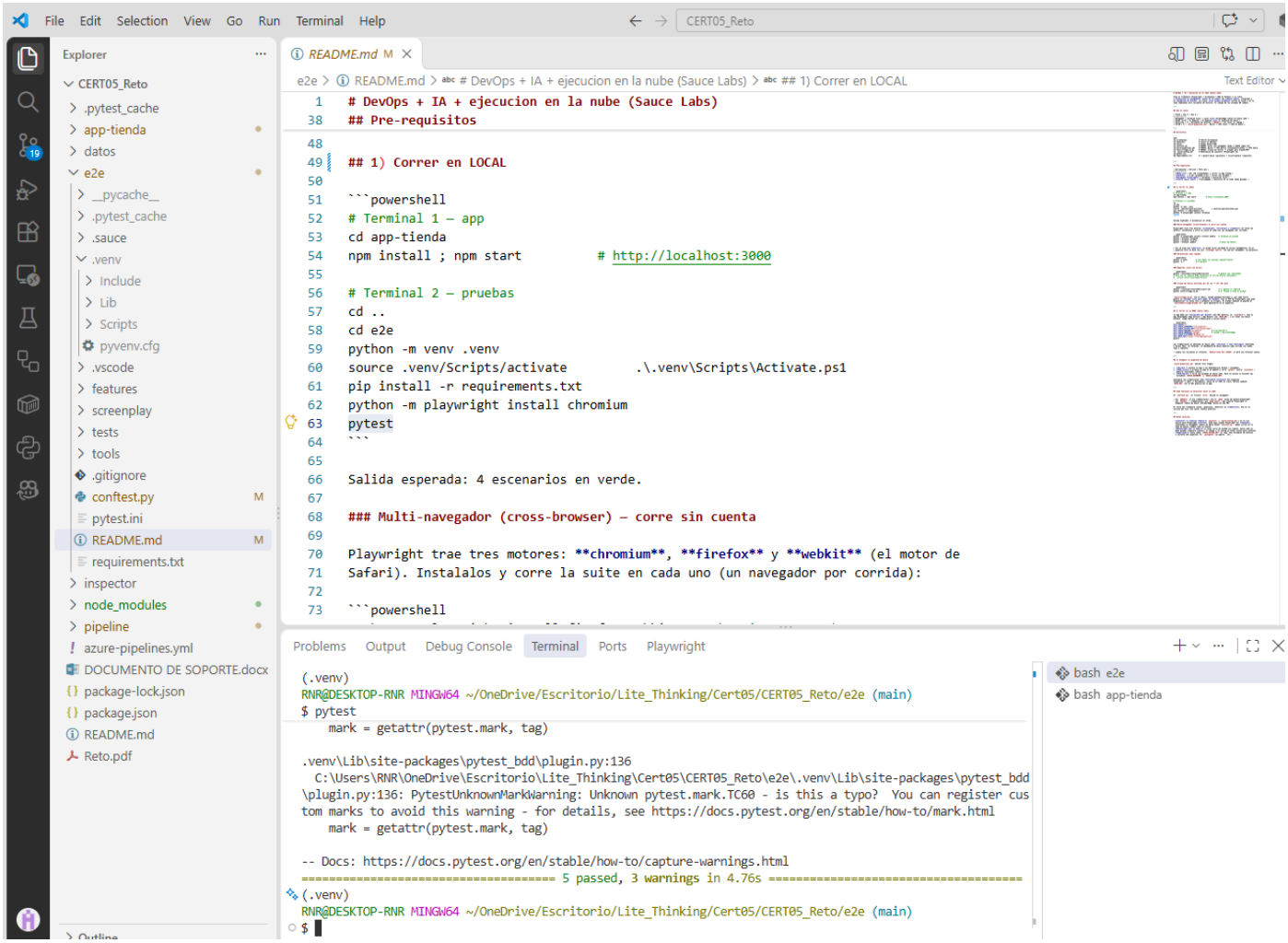
Cambios aplicados

1. Añadí `busqueda.feature` con el escenario Gherkin:

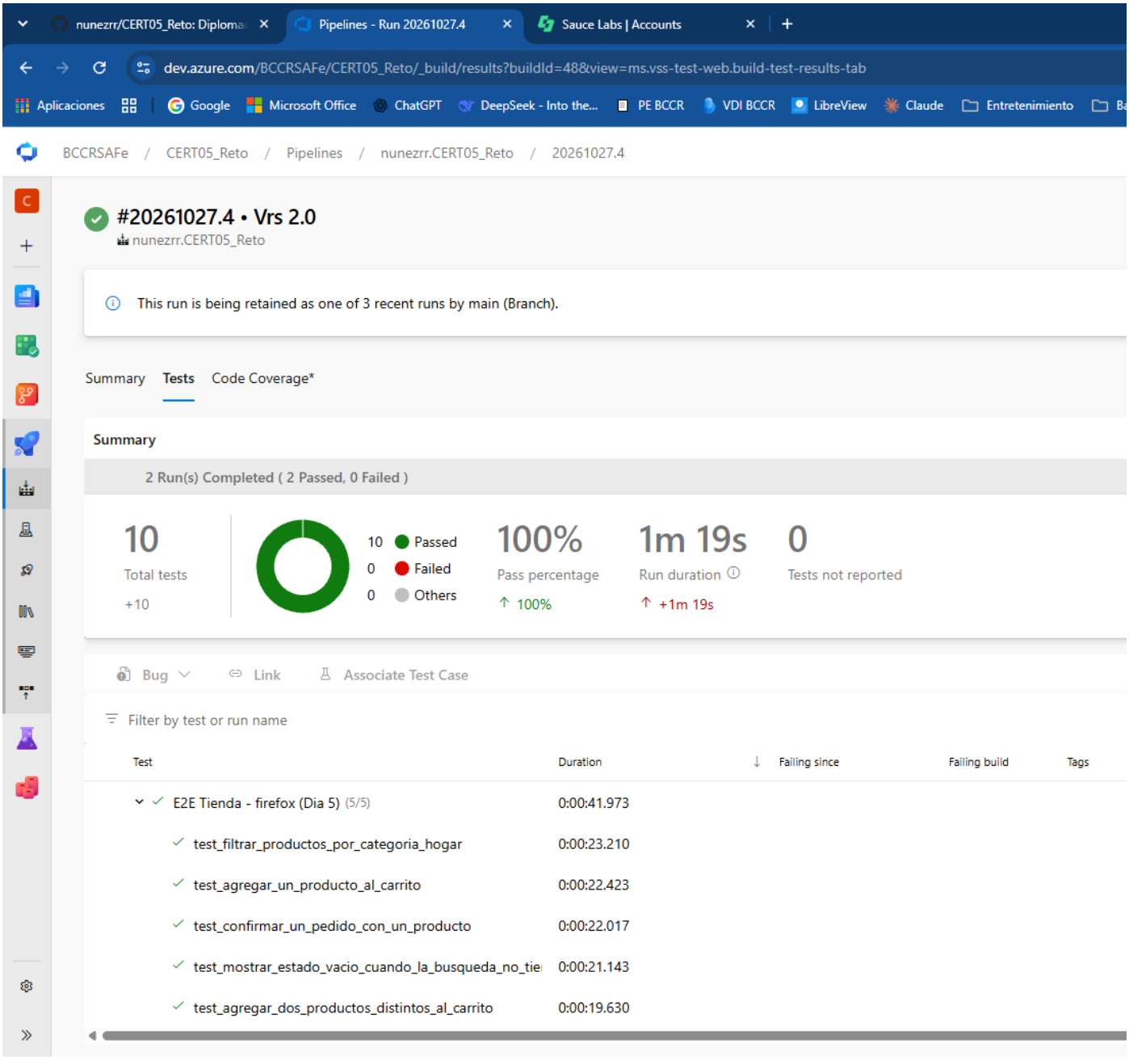
```
Scenario: Mostrar estado vacio cuando la busqueda no tiene coincidencias
  Given que abro la home de la tienda
  When busco el producto "zz-no-existe-qa-8472"
  Then no veo productos en el catalogo
  And veo el mensaje "No hay productos para ese filtro."
```

2. En `tasks.py`, agregué `BuscarProducto`, que escribe el término en el input existente `busqueda`.
3. En `questions.py`, agregué `MensajeSinResultados`, que obtiene el aviso visible. La cantidad de productos se verifica reutilizando `ProductosVisibles`.
4. En `steps_comunes.py`, añadí los steps de búsqueda, cero resultados y mensaje esperado.
5. Creé `test_busqueda.py` para enlazar el feature con pytest-bdd. No fue necesario modificar la aplicación: ya implementa el filtro y el estado vacío.

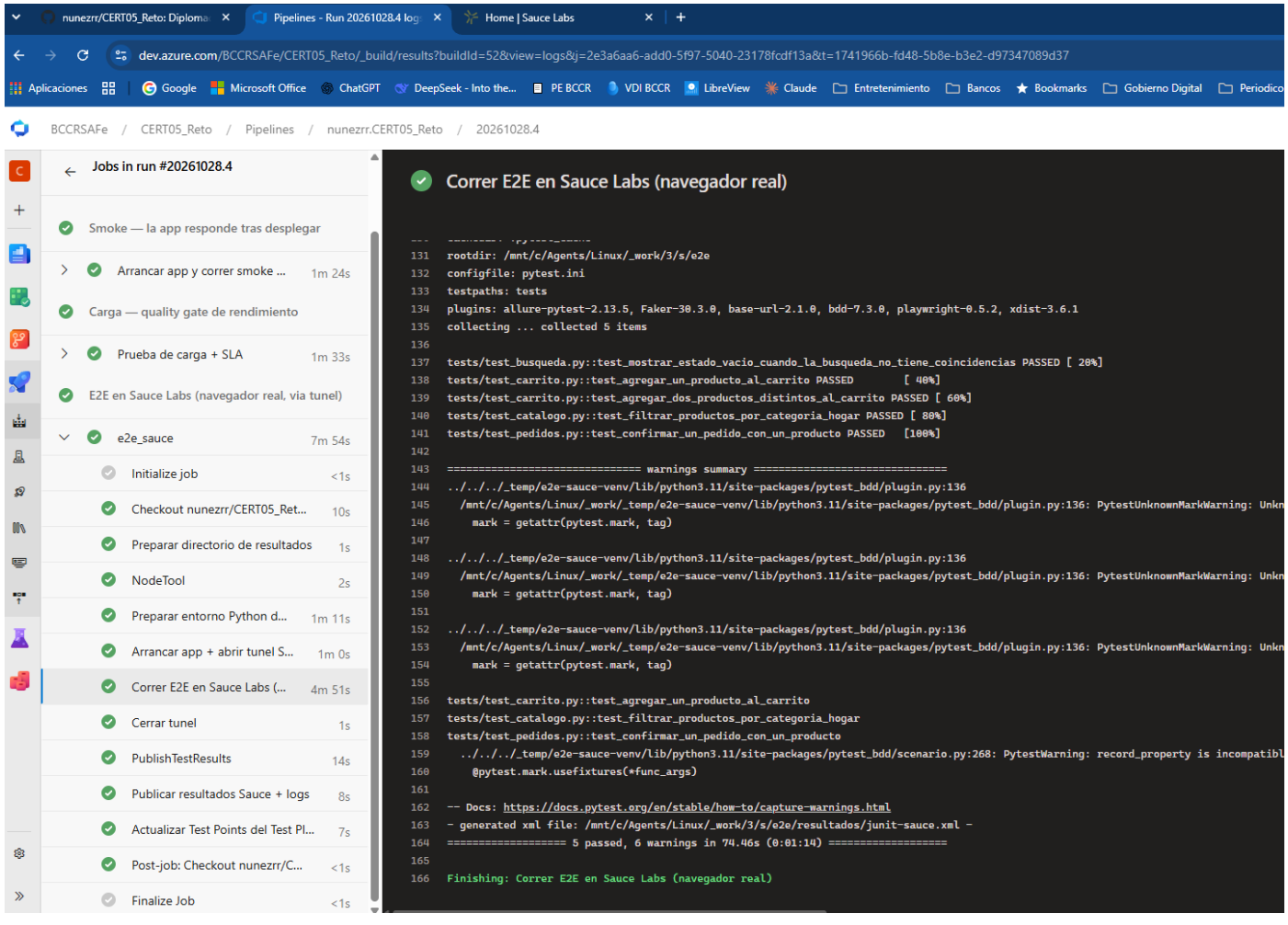
Validación: la suite completa pasó, `5 passed`. Quedan tres warnings existentes de pytest por los tags `@TC58`, `@TC59` y `@TC60`, no relacionados con este escenario.

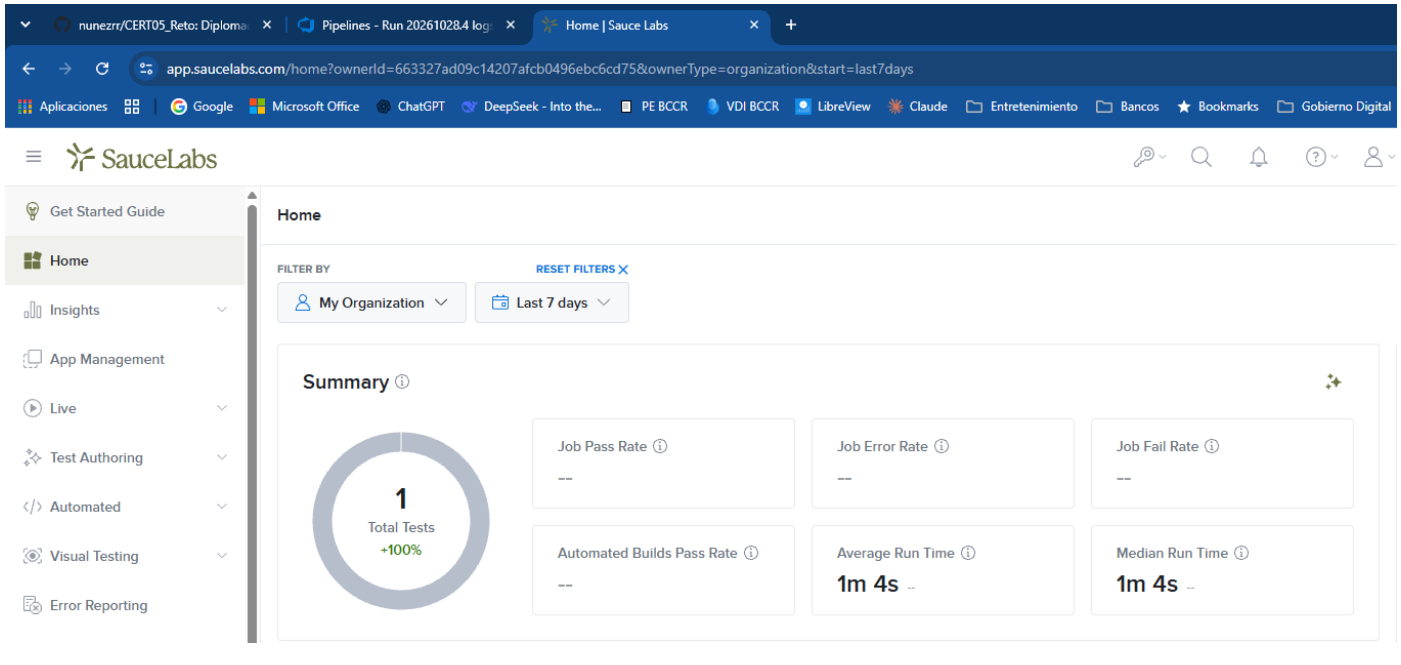


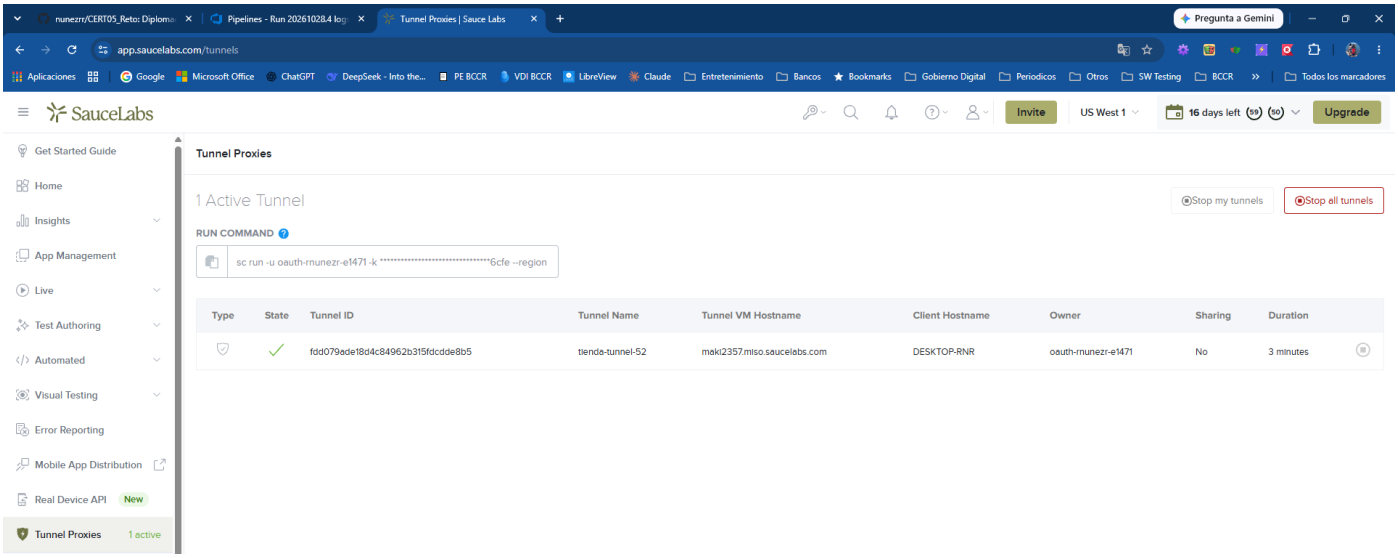
Uso Interno estricto


nunezrr/CERT05_Reto: Diploma
Pipelines - Run 20261027.4
Sauce Labs | Accounts
dev.azure.com/BCCRSAFe/CERT05_Reto/_build/results?buildId=48&view=ms.vss-test-web.build-test-results-tab
Aplicaciones
Google
Microsoft Office
ChatGPT
DeepSeek - Into the...
PE BCCR
VDI BCCR
LibreView
Claude
Entretenimiento
BCCRSAFe / CERT05_Reto / Pipelines / nunezrr.CERT05_Reto / 20261027.4
#20261027.4 • Vrs 2.0
nunezrr.CERT05_Reto
This run is being retained as one of 3 recent runs by main (Branch).
Summary
Tests
Code Coverage*
Summary
2 Run(s) Completed (2 Passed, 0 Failed)
10
Total tests
+10
10 Passed
0 Failed
0 Others
100%
Pass percentage
↑ 100%
1m 19s
Run duration
↑ +1m 19s
0
Tests not reported
Bug
Link
Associate Test Case
Filter by test or run name
Test
Duration
Failing since
Failing build
Tags
E2E Tienda - firefox (Dia 5) (5/5)
0:00:41.973
test_filtrar_productos_por_categoria_hogar
0:00:23.210
test_agregar_un_producto_al_carrito
0:00:22.423
test_confirmar_un_pedido_con_un_producto
0:00:22.017
test_mostrar_estado_vacio_cuando_la_busqueda_no_tie
0:00:21.143
test_agregar_dos_productos_distintos_al_carrito
0:00:19.630

Fase 4: configuración de ejecución en la nube + evidencia.

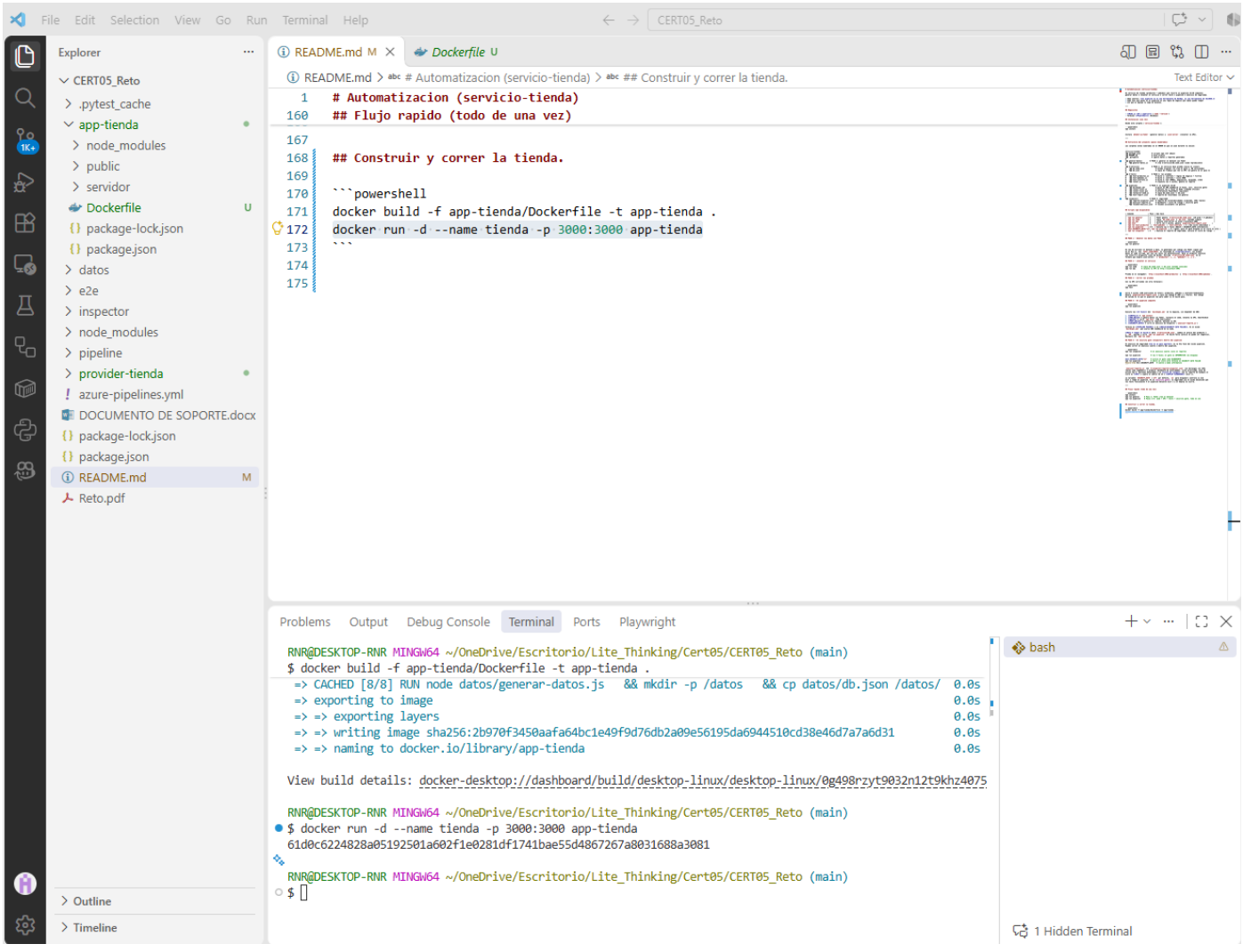


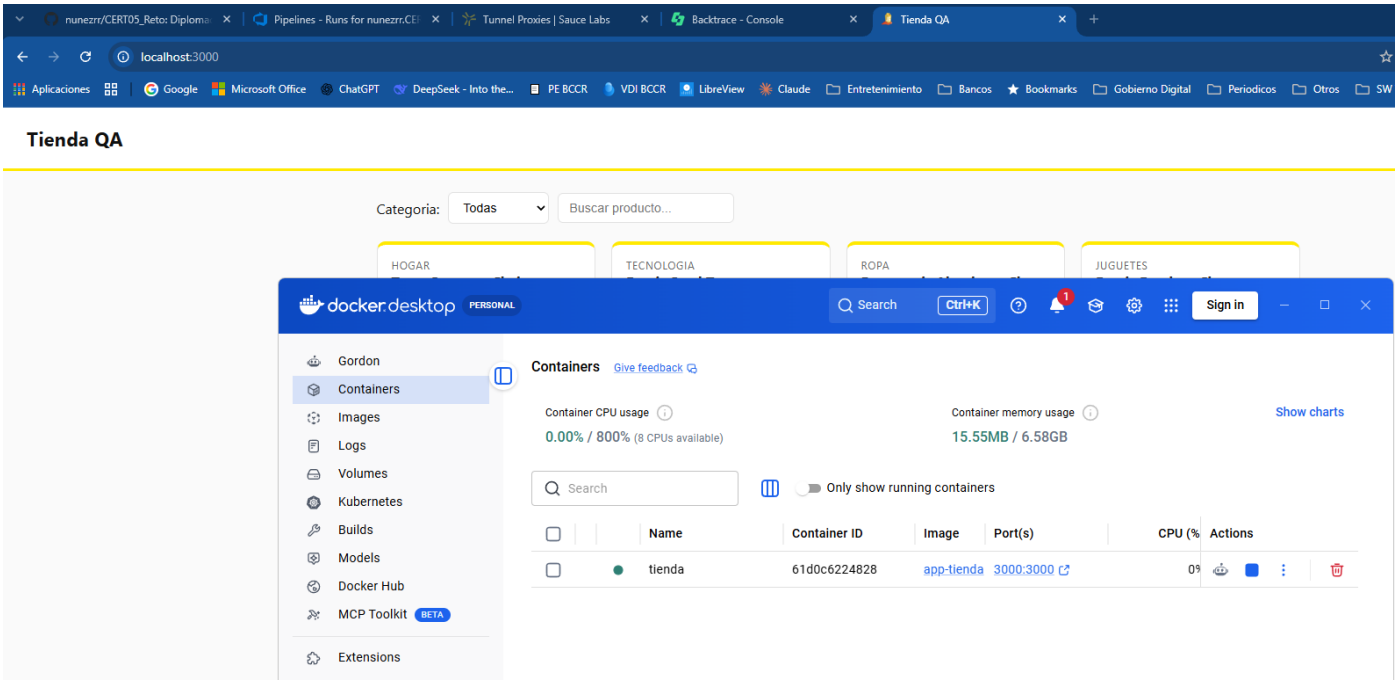


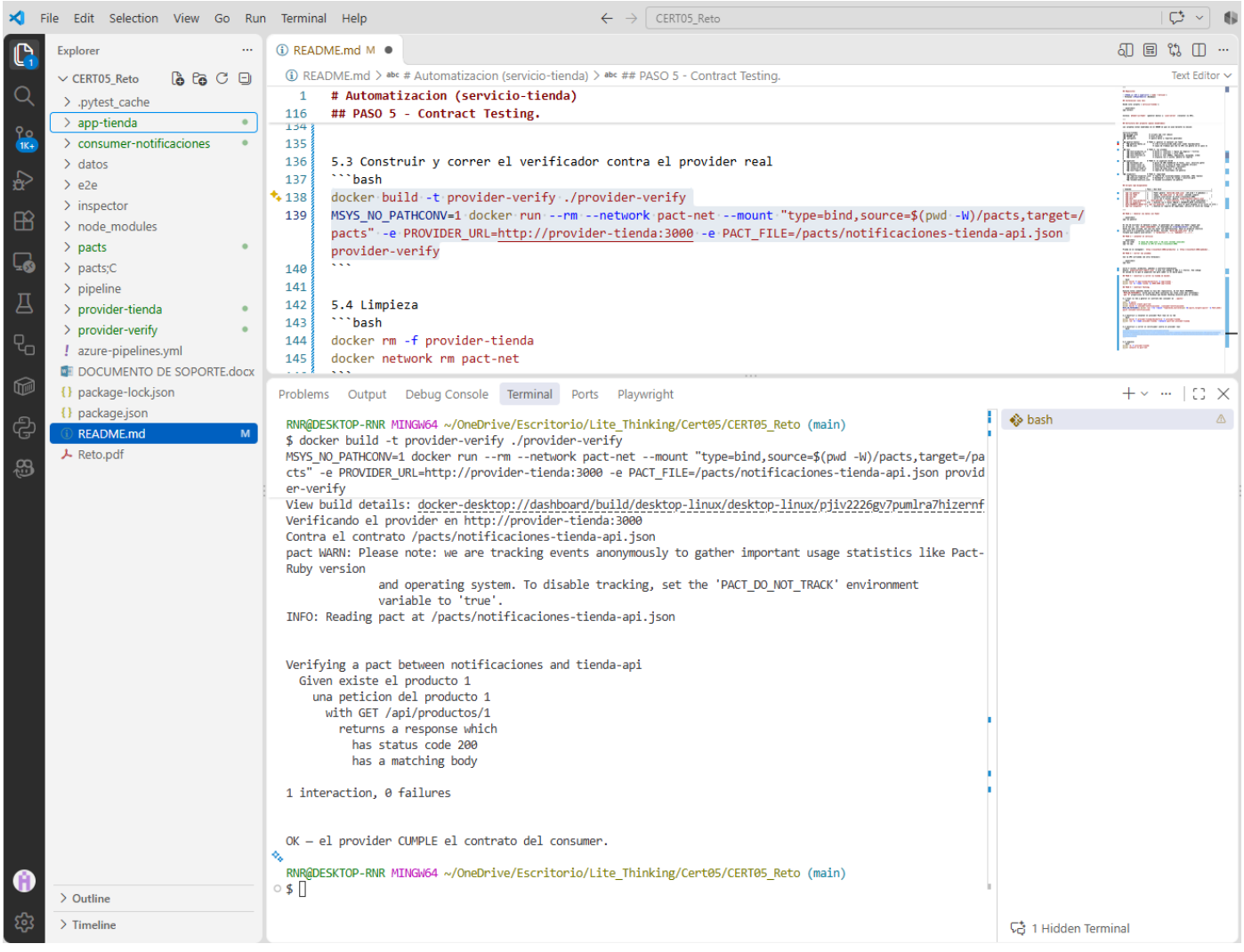


Uso Interno estricto

Fase 5: Dockerfile + imagen corriendo (y en ACR) + contract test (Pact) verificado.

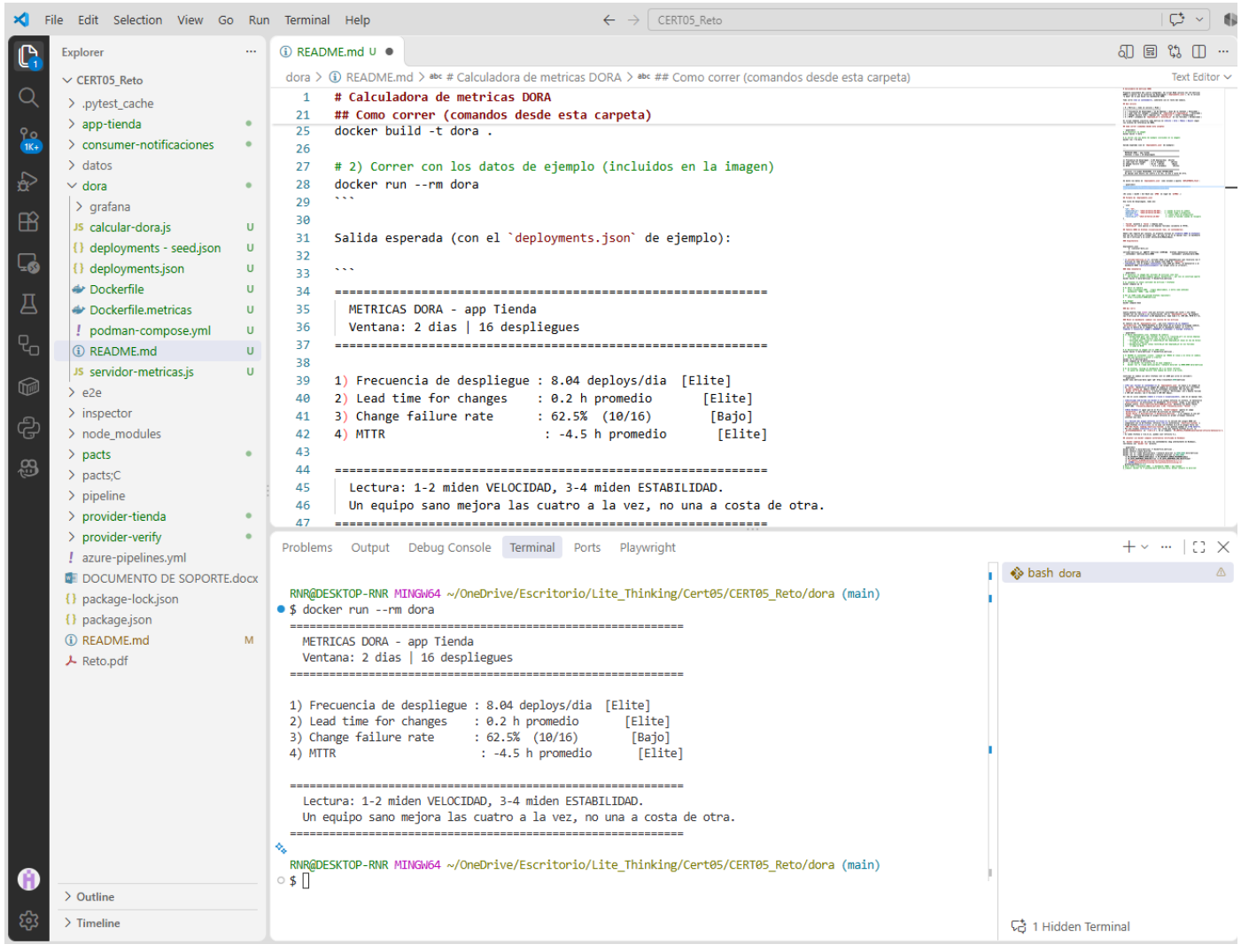


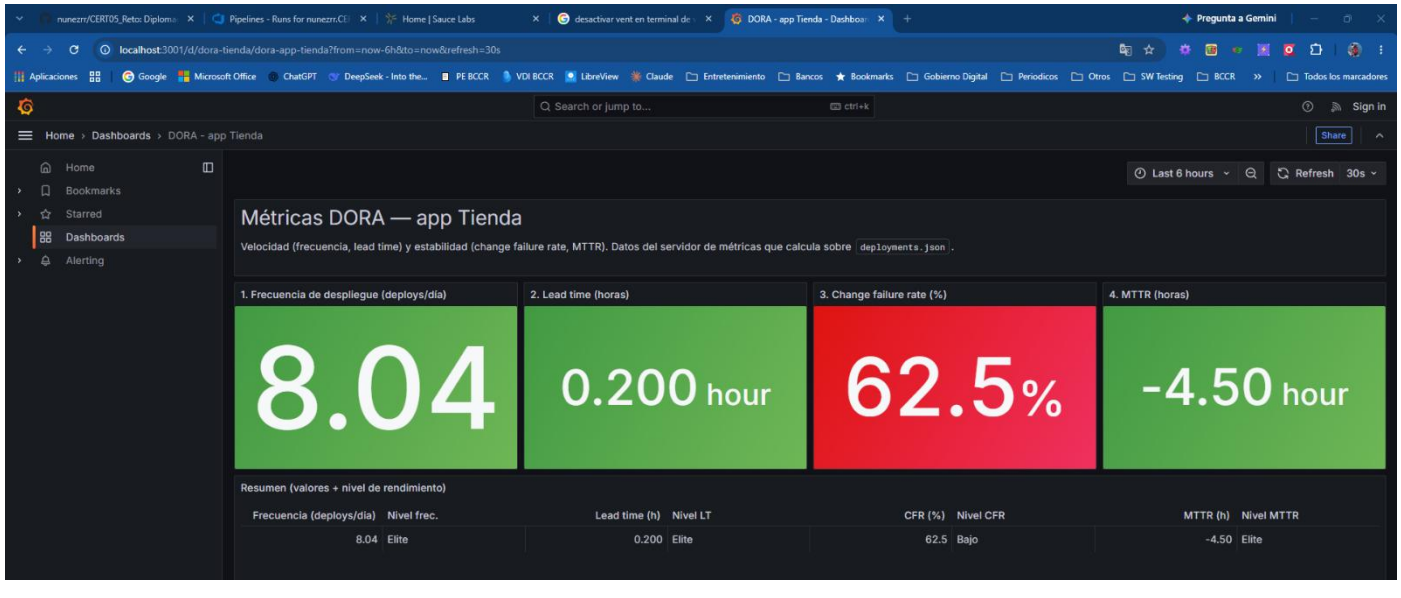


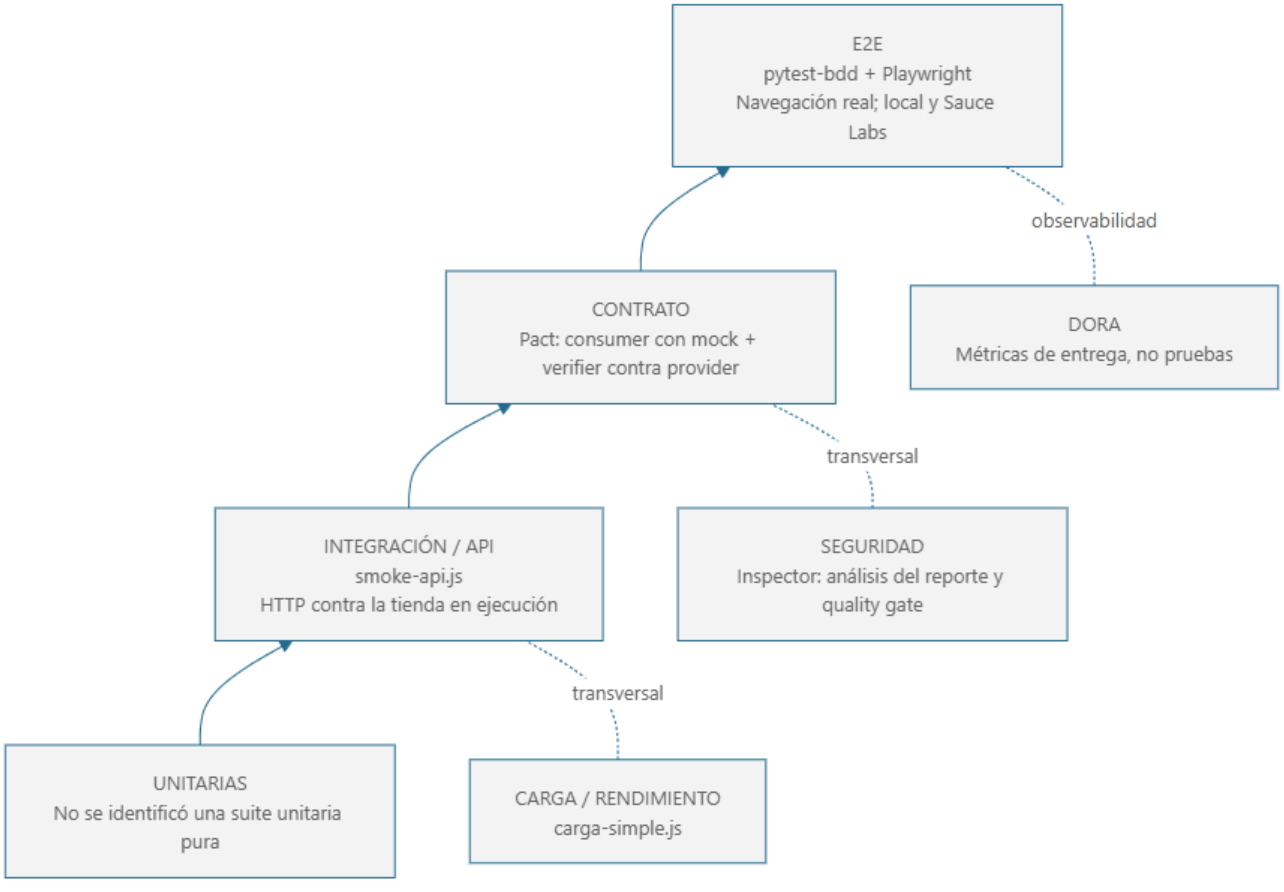


Uso Interno estricto

Fase 6: cuadro de métricas DORA + pirámide + riesgos y mitigaciones.







Por qué van ahí:

- **Integración/API:** smoke-api.js hace peticiones HTTP a la aplicación activa y verifica respuestas de salud, productos y pedidos; integra cliente y servicio, no prueba una función aislada.
- **Contrato:** test_consumer_pact.py valida al consumer contra el mock de Pact y genera el contrato. verificar_provider.py comprueba ese contrato contra el provider real. Está entre integración y E2E porque valida la frontera entre servicios sin recorrer toda la interfaz.
- **E2E:** los escenarios de tests usan pytest-bdd y Playwright para recorrer flujos como catálogo, búsqueda, carrito y pedidos en el navegador. pytest-bdd describe escenarios; Playwright automatiza el navegador. Sauce Labs cambia dónde corre el navegador, no el nivel de prueba.
- **Carga:** carga-simple.js mide latencias, tasa de errores y throughput con concurrencia. Es una prueba no funcional transversal, no un escalón clásico de la pirámide.
- **Seguridad:** Inspector analiza un reporte y aplica un gate; tampoco es un nivel funcional de la pirámide. **DORA** mide el desempeño de la entrega y necesita datos de despliegues/incidentes; no prueba el producto.

Riesgos:

1. **Cobertura unitaria insuficiente.** La solución tiene pruebas API, de contrato y E2E, pero no una suite clara de pruebas unitarias para la lógica de negocio.
   Mitigación: agregar pruebas unitarias aisladas para cálculos, validaciones de pedidos y otras reglas de negocio.
2. **Contrato Pact con cobertura limitada.** El consumer verifica principalmente el caso exitoso de obtener un producto; otros escenarios de API podrían cambiar sin que el contrato los detecte.
   Mitigación: añadir interacciones para errores y casos límite, y ejecutar la verificación Pact automáticamente en cada cambio.
3. **Pruebas frágiles por diferencias de entorno.** El pipeline depende de herramientas y servicios externos, como Python en el agente, Docker y Sauce Labs.
   Mitigación: estandarizar versiones, validar dependencias y credenciales al inicio, y separar fallos de infraestructura de fallos de la aplicación.